



Angular 20 完整学习指南

这是一份全面、系统、图文并茂的 Angular 20 框架深度学习资料，旨在帮助开发者从基础入门到高级应用。



目录结构

- [第一部分：核心基础](#)
- [第二部分：高级特性](#)
- [第三部分：工程实践](#)
- [第四部分：性能优化](#)
- [第五部分：面试题汇总](#)

第一部分：核心基础



什么是 Angular?



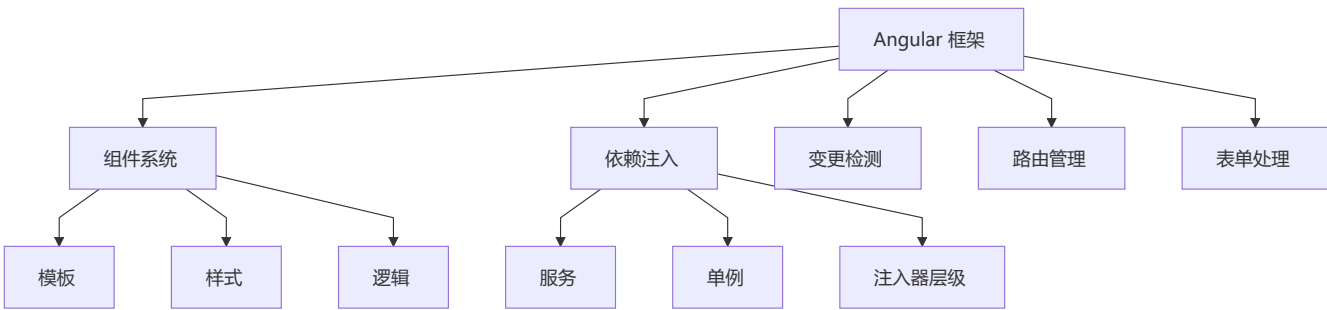
核心定义

Angular 是由 Google 开发的开源、企业级 TypeScript 框架，用于构建高性能、可维护的**单页面应用 (SPA)**。

```
// Angular 的三大特性：  
// 1. 基于 TypeScript：强类型，开发时捕获错误  
// 2. 组件化架构：模块化、可复用的 UI 构件  
// 3. 完整的框架：内置路由、表单、HTTP、测试等
```



Angular 的核心角色



Angular vs 其他框架

特性	Angular	React	Vue
类型系统	✅ TypeScript 原生	❌ 需第三方库	⚠️ 部分支持
学习曲线	🔴 陡峭	🟡 中等	🟢 平缓

特性	Angular	React	Vue
企业应用	✅ 完美	✅ 良好	⚠️ 可行
包大小	🔴 较大	🟡 中等	🟢 较小
内置工具	✅ 完整	⚠️ 需组合	⚠️ 部分集成

2 Angular 20 新特性详解

🌟 重要特性速览

Angular 20 (2024)

- └ Signals 生产级发布
- └ 新控制流语法 (@if/@for/@switch)
- └ 延迟加载块 (@defer)
- └ 更新的 HTTP 客户端
- └ Zoneless 检测模式
- └ 独立组件默认生成

🔗 Signals 响应式系统详解

问题背景

在 Angular 18 之前，检测变化需要遍历整个组件树：

变更发生 → Zone.js 拦截 → 整个树遍历 → 每个组件 detectChanges

这在大型应用中会导致性能问题。

解决方案：Signal

Signals 提供**细粒度的反应性**：

```
import { signal, computed, effect } from '@angular/core';

// 📍 创建可写信号
const count = signal(0);

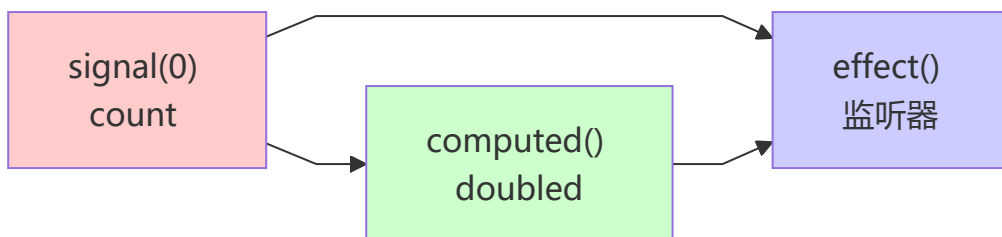
// 📍 派生计算信号（自动依赖追踪）
const doubled = computed(() => count() * 2);
const message = computed(() => {
  const c = count();
  return c === 0 ? '零' : c === 1 ? '一' : `${c}个`;
});

// 📍 监听变化副作用
effect(() => {
  console.log(`Count 变化: ${count()}`);
  console.log(`Doubled: ${doubled()}`);
});
```

```
});

// 📌 更新信号
count.set(5);           // 直接赋值
count.update(v => v+1); // 基于旧值更新
```

执行流程图



✨ 新控制流语法

✗ 旧方式 vs ✔ 新方式对比

```

<!-- 旧方式: 指令风格 -->
<div *ngIf="isLoading" class="spinner"></div>
<div *ngIf="!isLoading" class="content">
  <div *ngFor="let item of items; trackBy: trackById">
    {{ item.name }}
  </div>
</div>

<!-- ✨ 新方式: 块级语法 -->
@if (isLoading) {
  <div class="spinner">加载中...</div>
} @else {
  <div class="content">
    @for (item of items; track item.id) {
      <div>{{ item.name }}</div>
    }
  </div>
}

```

改进点:

- ✔ 语法更清晰
- ✔ 自动 `trackBy` 支持
- ✔ 编译器优化更好
- ✔ 性能提升 20-30%

延迟加载块 (@defer)

```
@Component({
  selector: 'app-dashboard',
  template: `
    <!-- 立即加载 -->
    <app-header></app-header>

    <!-- 延迟加载: 当进入视口时 -->
    @defer (on viewport) {
      <app-heavy-chart></app-heavy-chart>
    } @placeholder {
      <div>图表加载中...</div>
    }

    <!-- 延迟加载: 交互时 -->
    @defer (on interaction) {
      <app-comments-section></app-comments-section>
    } @loading {
      <p>评论加载中...</p>
    }

    <!-- 延迟加载: 条件满足时 -->
    @defer (when isPremiumUser()) {
      <app-premium-features></app-premium-features>
    }
  `,
})
export class DashboardComponent {
  isPremiumUser = signal(false);
}
```

性能收益:

- 初始加载体积减少 40-50%
- 首屏加载时间缩短
- 按需加载组件和组件逻辑

TypeScript 与 Angular 深度融合

装饰器系统 (Decorators)

装饰器是 Angular 的核心，它为类、属性、方法添加元数据：

```
// 📌 类装饰器
@Component({
  selector: 'app-hero',
  template: `...`,
  styles: [`...`],
  standalone: true,
```

```

    changeDetection: ChangeDetectionStrategy.OnPush
  })
  export class HeroComponent { }

  // 📌 属性装饰器
  export class ChildComponent {
    @Input() heroName!: string;
    @Input() set age(value: number) { /* getter */ }
    @Output() heroSelected = new EventEmitter<Hero>();

    @ViewChild(CharComponent) chart!: CharComponent;
    @ViewChildren(ListItemComponent) items!: QueryList<ListItemComponent>;
    @ContentChild(ActionBarComponent) actionBar!: ActionBarComponent;
  }

  // 📌 方法装饰器（如果在库中）
  @HostListener('click', ['$event'])
  onClick(event: MouseEvent) { }

  // 📌 参数装饰器
  constructor(@Inject(DOCUMENT) doc: Document) { }

```

类型安全的组件

```

// ✅ 正确：强类型的 Product 接口
interface Product {
  id: number;
  name: string;
  price: number;
  rating?: number;
  tags: string[];
}

@Component({
  selector: 'app-product-list',
  template: `
    @for (product of products(); track product.id) {
      <app-product-card
        [product]="product"
        (onSelect)="onProductSelect($event)"
      />
    }
  `,
  standalone: true,
  imports: [CommonModule, ProductCardComponent]
})
export class ProductListComponent {
  // signal 类型约束
  products = signal<Product[]>([]);
  selectedProduct = signal<Product | null>(null);

  private productService = inject(ProductService);

```

```

ngOnInit() {
  // 类型检查: productService.getProducts() 返回 Observable<Product[]>
  this.productService.getProducts().subscribe(
    products => this.products.set(products)
  );
}

onProductSelect(product: Product): void {
  this.selectedProduct.set(product);
}
}

```

4 组件系统深层理解

✂ 组件解剖

```

import { Component, Input, Output, EventEmitter, signal, computed } from '@angular/core';
import { CommonModule } from '@angular/common';

interface TodoItem {
  id: number;
  text: string;
  completed: boolean;
}

@Component({
  selector: 'app-todo-list',
  standalone: true,
  imports: [CommonModule],
  template: `
    <!-- 1 模板: 定义视图 -->
    <div class="todo-container">
      <h2>{{ title }}</h2>

      @for (todo of displayedTodos(); track todo.id) {
        <div
          class="todo-item"
          [class.completed]="todo.completed"
          (click)="toggleTodo(todo.id)"
        >
          <span>{{ todo.text }}</span>
          <button (click)="removeTodo(todo.id); $event.stopPropagation()">
            删除
          </button>
        </div>
      }

      <div class="stats">
        已完成: {{ completedCount() }} / 总数: {{ todos().length }}
      </div>
    </div>
  `
})

```

```

    </div>
  `
  ,
  // 2 样式: 组件作用域样式
  styles: [ `
    .todo-container {
      max-width: 500px;
      margin: 20px auto;
    }
    .todo-item {
      padding: 10px;
      border: 1px solid #ddd;
      margin: 5px 0;
      cursor: pointer;
      display: flex;
      justify-content: space-between;
    }
    .todo-item.completed {
      text-decoration: line-through;
      opacity: 0.5;
    }
  `
  ]
})
export class TodoListComponent {
  // 3 数据: 响应式状态管理
  @Input() title: string = '我的任务列表';
  @Output() todoAdded = new EventEmitter<TodoItem>();

  todos = signal<TodoItem[]>([
    { id: 1, text: '学习 Angular', completed: false },
    { id: 2, text: '完成项目', completed: false }
  ]);

  // 4 计算属性: 派生状态
  completedCount = computed(() =>
    this.todos().filter(t => t.completed).length
  );

  displayedTodos = computed(() =>
    this.todos().filter(t => !t.completed)
  );

  // 5 方法: 处理逻辑
  toggleTodo(id: number): void {
    this.todos.update(todos =>
      todos.map(t =>
        t.id === id ? { ...t, completed: !t.completed } : t
      )
    );
  }

  removeTodo(id: number): void {
    this.todos.update(todos => todos.filter(t => t.id !== id));
  }
}

```

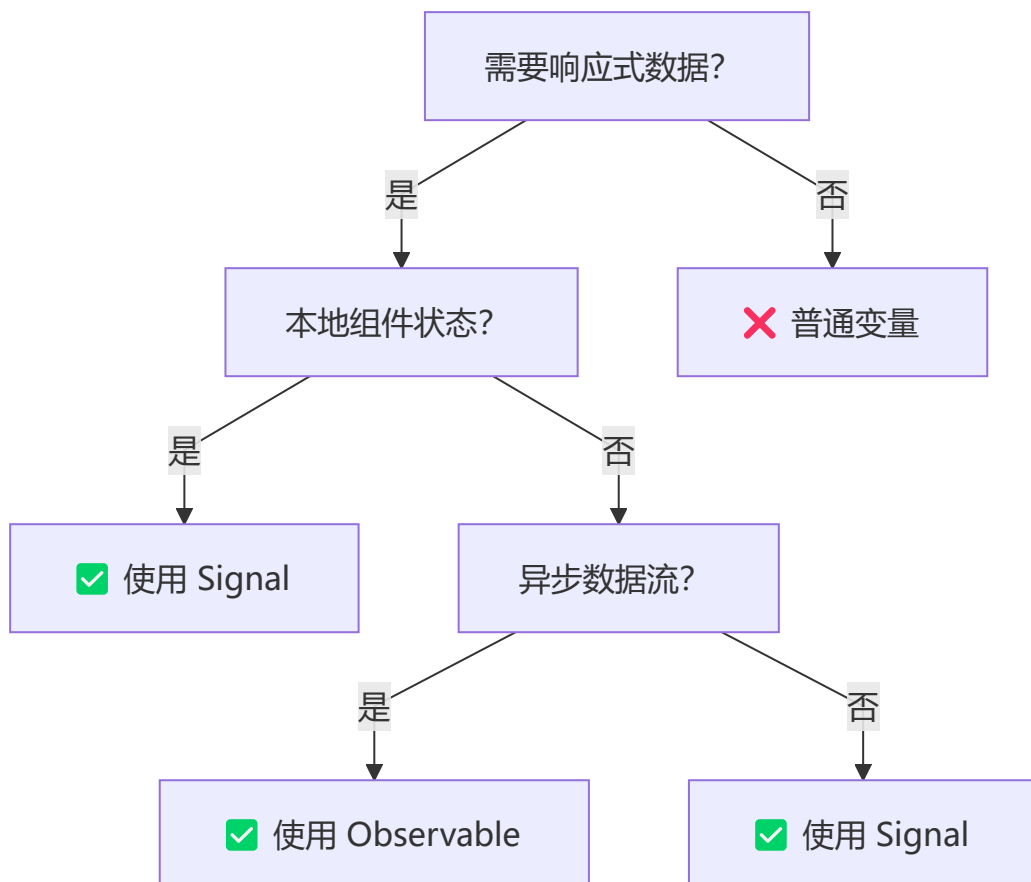
```
}
```

📄 模板语法完整参考

语法	用途	示例
<code>{{ expression }}</code>	插值	<code>{{ user.name }}</code>
<code>[property]="value"</code>	属性绑定	<code>[disabled]="!form.valid"</code>
<code>(event)="handler()"</code>	事件绑定	<code>(click)="submit()"</code>
<code>[(ngModel)]="value"</code>	双向绑定	<code>[(ngModel)]="searchTerm"</code>
<code>@if (condition)</code>	条件渲染	<code>@if (isAdmin) { ... }</code>
<code>@for (item of list)</code>	列表渲染	<code>@for (item of items; track item.id)</code>
<code>\ pipe</code>	管道转换	<code>{{ price \ currency }}</code>

5 Signals vs Observables

🤔 何时使用哪一个？



详细对比

```
// 场景 1: 本地组件状态 → signal 更好
const userCount = signal(0);
const users = computed(() => allUsers().slice(0, userCount()));

// 场景 2: HTTP 请求 → 两者都可, signal 推荐
// 方式 A: Observable (需要手动管理订阅)
users$ = this.http.get('/users');

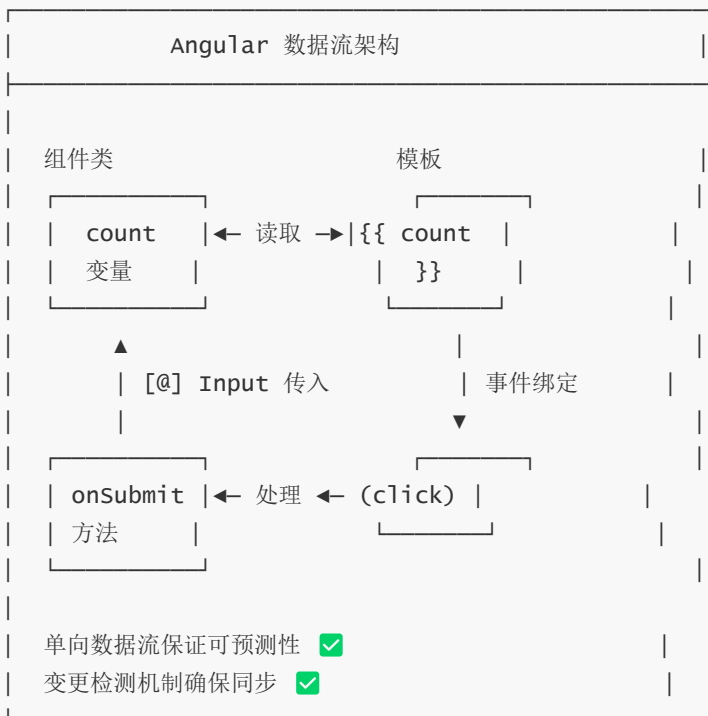
// 方式 B: signal (推荐, 更现代)
users = resource(() => ({
  request: { /* 参数 */ },
  loader: ({ request }) => this.http.get('/users')
}));

// 场景 3: 事件流、轮询 → Observable 更好
const messages$ = this.messageService.getMessages().pipe(
  switchMap(msg => this.processMessage(msg))
);

// 场景 4: WebSocket 连接 → Observable 最优
socket$ = websocket('ws://...');
```

6 数据绑定深度剖析

数据流向可视化



四种绑定方式详解

```
<!-- 1 插值绑定: 组件 → 模板 -->
<h1>{{ title }}</h1>

<!-- 2 属性绑定: 组件 → DOM属性 -->
<img [src]="imageUrl" [alt]="imageName" />
<button [disabled]="isSubmitting">提交</button>

<!-- 3 事件绑定: 模板 → 组件 -->
<button (click)="onSave()">保存</button>
<input (keyup.enter)="search()" placeholder="搜索...">

<!-- 4 双向绑定: 组件 ↔ 模板 -->
<input [(ngModel)]="username" />
<!-- 等价于 -->
<input
  [ngModel]="username"
  (ngModelChange)="username = $event"
/>
```

高级绑定技巧

```
<!-- 事件对象 -->
<input (keyup)="onKeyUp($event)" />

<!-- 模板变量 -->
<input #nameInput type="text" />
<button (click)="greet(nameInput.value)">问候</button>

<!-- 按键事件修饰符 -->
<input (keyup.enter)="save()" />      <!-- Enter 键 -->
<input (keyup.escape)="cancel()" />    <!-- Esc 键 -->

<!-- 鼠标事件修饰符 -->
<button (mouseenter)="highlight()" (mouseleave)="unhighlight()">
  悬停
</button>

<!-- 事件停止冒泡 -->
<div (click)="onParentClick()">
  <button (click)="onChildClick(); $event.stopPropagation()">
    内层按钮
  </button>
</div>
```

7 RxJS 在 Angular 中的应用

Observable 核心概念

```
import { Observable, Subject, BehaviorSubject, ReplaySubject } from 'rxjs';
import { map, filter, debounceTime, distinctUntilChanged, switchMap, tap, catchError } from
  'rxjs/operators';

// 📍 创建 Observable 的多种方式

// 方式 1: from 创建
from([1, 2, 3]).subscribe(console.log);

// 方式 2: timer 创建
timer(1000, 2000).subscribe(() => console.log('每2秒触发'));

// 方式 3: 创建可观察的 HTTP 请求
const users$ = this.http.get<User[]>('/api/users');

// 方式 4: Subject - 可观察对象和观察者的混合体
const userClick$ = new Subject<ClickEvent>();
userClick$.subscribe(event => console.log('用户点击了'));
userClick$.next(clickEvent); // 发出新值
```

常用操作符详解

```
// 1 转换操作符
source$.pipe(
  map(x => x * 2), // 变换每个值
  switchMap(x => this.fetch(x)) // 切换到新 observable
);

// 2 过滤操作符
source$.pipe(
  filter(x => x > 10), // 过滤值
  distinctUntilChanged() // 去重相邻值
);

// 3 时间操作符
source$.pipe(
  debounceTime(300), // 防抖（最后一个事件）
  throttleTime(1000) // 节流（固定间隔）
);

// 4 组合操作符
combineLatest([users$, posts$]).pipe(
  map(([users, posts]) => ({ users, posts }))
);

// 5 错误处理
source$.pipe(
```

```
    retry(3), // 重试3次
    catchError(err => of(defaultValue)) // 捕获错误
  );
```

实战场景：搜索输入框

```
@Component({
  selector: 'app-search',
  template: `
    <input
      [formControl]="searchControl"
      placeholder="搜索用户..."
    />

    @for (result of results$ | async) {
      <div class="result">{{ result.name }}</div>
    }
  `,
})
export class SearchComponent implements OnInit {
  searchControl = new FormControl('');
  results$: Observable<User[]>;

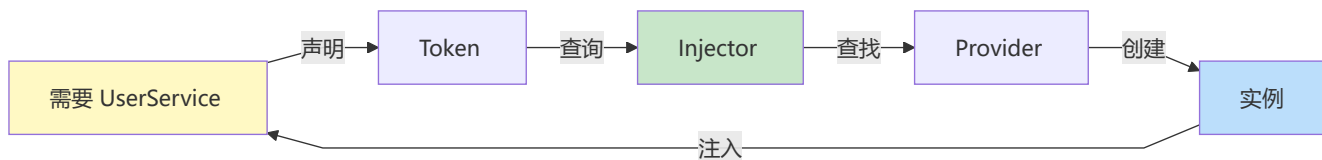
  constructor(private userService: UserService) {}

  ngOnInit() {
    this.results$ = this.searchControl.valueChanges.pipe(
      // 第一步：等待用户停止输入
      debounceTime(300),
      // 第二步：过滤重复搜索词
      distinctUntilChanged(),
      // 第三步：过滤空值
      filter(term => term.length > 0),
      // 第四步：发起 HTTP 请求
      switchMap(term => this.userService.search(term)),
      // 第五步：错误处理
      catchError(error => {
        console.error('搜索失败', error);
        return of([]);
      })
    );
  }
}
```

第二部分：高级特性

8 依赖注入 (DI) 系统

DI 核心原理



Provider 提供者详解

```
import { Injectable, inject, InjectionToken } from '@angular/core';

// 1 服务提供者（最常见）
@Injectable({ providedIn: 'root' })
export class UserService {
  users = signal<User[]>([]);

  getUsers() { /* ... */ }
}

// 2 值提供者
const appConfig = new InjectionToken<AppConfig>('app.config');
const configProvider = {
  provide: appConfig,
  useValue: { apiUrl: 'https://api.example.com' }
};

// 3 类提供者
const httpProvider = {
  provide: HttpClient,
  useClass: CachedHttpClient // 使用子类替代
};

// 4 工厂提供者
const dateProvider = {
  provide: 'app.timestamp',
  useFactory: () => new Date().getTime()
};

// 5 注入令牌（提供非类型的依赖）
export const API_URL = new InjectionToken<string>('api.url');
export const DATABASE = new InjectionToken('app.database');

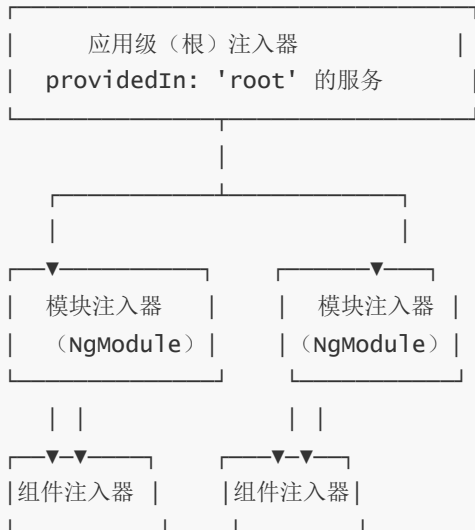
@Injectable()
export class DataService {
  constructor(
    @Inject(API_URL) private apiUrl: string,
```

```

@Inject(DATABASE) private db: Database
) {}
}

```

注入器层级结构



现代 DI 用法 (inject() API)

```

// ✅ 推荐：使用 inject() 的函数式方式
@Component({
  selector: 'app-user',
  standalone: true
})
export class UserComponent {
  // 在组件类中直接使用
  private userService = inject(UserService);
  private route = inject(ActivatedRoute);
  private apiUrl = inject(API_URL);

  ngOnInit() {
    this.userService.getUsers();
  }
}

// 📍 在函数/管道中也能使用
export function loadUserGuard() {
  const userService = inject(UserService);
  const router = inject(Router);

  return () => userService.isLoading() || router.navigate(['/login']);
}

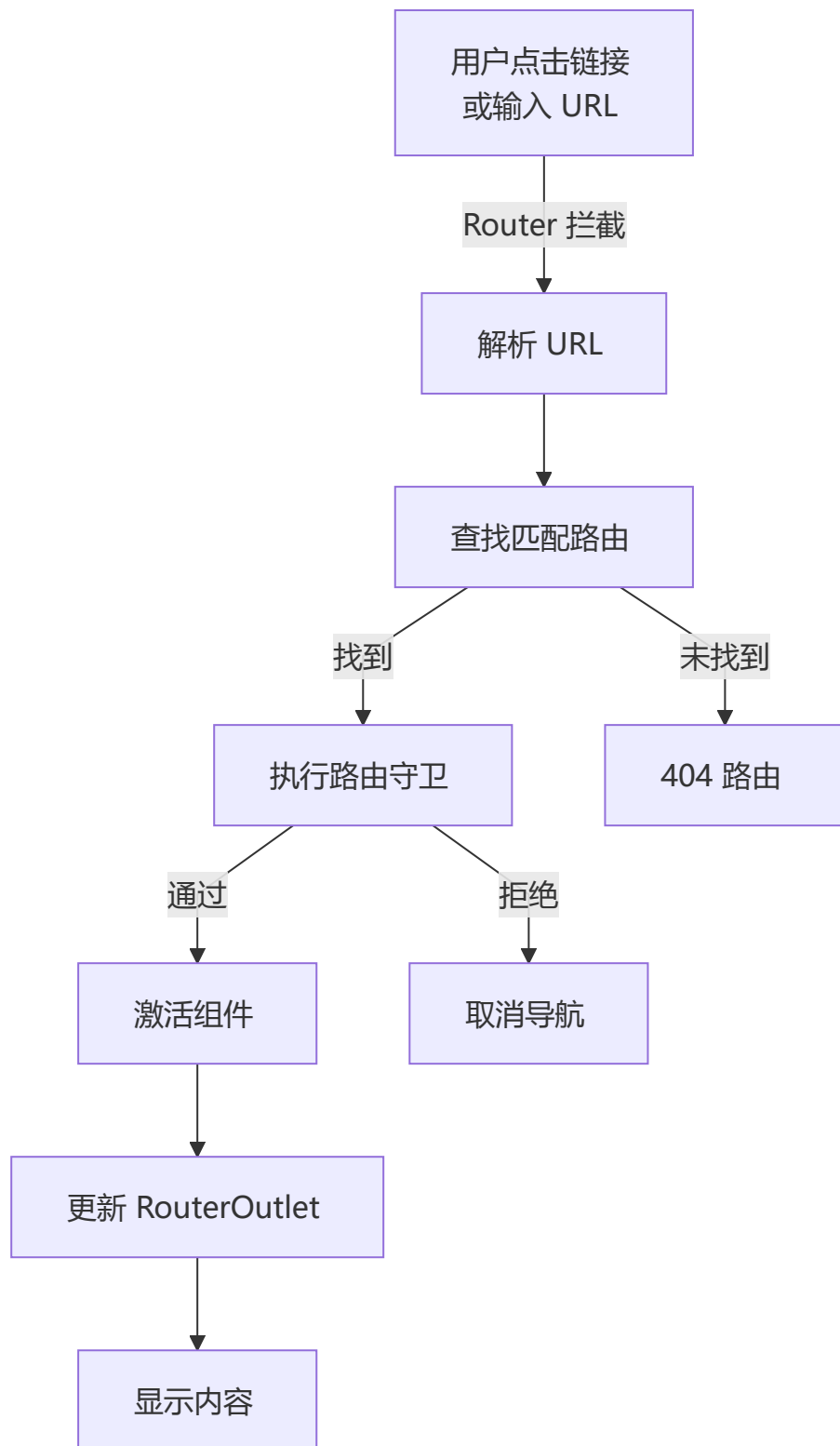
export class UppercasePipe implements PipeTransform {
  private logger = inject(LogService);

```

```
transform(value: string): string {  
  this.logger.log(`Transforming: ${value}`);  
  return value.toUpperCase();  
}  
}
```

9 路由系统 (Router)

📌 路由工作流程



路由配置详细示例

```
import { Routes, Router, ActivatedRoute } from '@angular/router';
import { inject } from '@angular/core';

// 📍 路由守卫示例
export function authGuard(): boolean {
  const authService = inject(AuthService);
  const router = inject(Router);
```



```

if (authService.isAuthenticated()) {
  return true;
} else {
  router.navigate(['/login']);
  return false;
}
}

// 📍 路由解析器（预加载数据）
export function userResolver() {
  return (route: ActivatedRouteSnapshot) => {
    const userId = route.paramMap.get('id');
    return inject(UserService).getUserById(userId!);
  };
}

// 📍 完整的路由配置
export const routes: Routes = [
  // 1 简单路由
  { path: '', redirectTo: '/dashboard', pathMatch: 'full' },

  // 2 组件路由
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [authGuard], // 进入前守卫
    canDeactivate: [unsavedChangesGuard] // 离开前守卫
  },

  // 3 参数路由
  {
    path: 'user/:id',
    component: UserDetailComponent,
    resolve: { user: userResolver() } // 预加载数据
  },

  // 4 嵌套路由（子路由）
  {
    path: 'admin',
    component: AdminLayoutComponent,
    canActivate: [adminGuard],
    children: [
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
      { path: 'dashboard', component: AdminDashboardComponent },
      { path: 'users', component: AdminUsersComponent },
      { path: 'settings', component: SettingsComponent }
    ]
  },

  // 5 延迟加载模块
  {
    path: 'analytics',

```

```

loadChildren: () =>
  import('./analytics/analytics.module').then(m => m.AnalyticsModule),
  canLoad: [AuthGuard]
},

// 6 通配符路由（必须放在最后）
{ path: '**', component: NotFoundComponent }
];

// 在组件中使用路由
@Component({
  selector: 'app-user-detail',
  template: `
    <h1>User: {{ user?.name }}</h1>
    <p>ID: {{ userId }}</p>
    <button (click)="goBack()">返回</button>
  `
})
export class UserDetailComponent {
  private router = inject(Router);
  private route = inject(ActivatedRoute);

  userId = '';
  user: User | null = null;

  ngOnInit() {
    // 方式 1: 从 resolve 获取数据
    this.user = this.route.snapshot.data['user'];

    // 方式 2: 从参数获取
    this.userId = this.route.snapshot.paramMap.get('id') || '';

    // 方式 3: 监听参数变化（当组件复用时）
    this.route.params.subscribe(params => {
      this.userId = params['id'];
      this.loadUser();
    });
  }

  goBack() {
    this.router.navigate(['../'], { relativeTo: this.route });
  }
}

```

声明式导航

```

<!-- 基础导航 -->
<a routerLink="/dashboard">仪表板</a>

<!-- 带参数 -->
<a [routerLink]="['/user', userId]">查看用户</a>

```

```

<!-- 查询参数 -->
<a [routerLink]="['/search']" [queryParams]="{ q: 'angular' }">
  搜索 Angular
</a>

<!-- 活跃链接标记 -->
<nav>
  <a routerLink="/home" routerLinkActive="active">首页</a>
  <a routerLink="/about" routerLinkActive="active"
    [routerLinkActiveOptions]="{ exact: true }">
    关于
  </a>
</nav>

<!-- 路由出口 -->
<div class="container">
  <router-outlet></router-outlet>
</div>

<!-- 多个路由出口 -->
<router-outlet></router-outlet>
<router-outlet name="sidebar"></router-outlet>

```

10 表单系统深度剖析



表单类型选择指南

表单类型选择

- |
- ├ 简单表单? (< 5 个字段)
 - | └ ☒ 模板驱动表单
- |
- ├ 复杂/动态表单?
 - | └ ☒ 响应式表单
- |
- ├ 需要自定义验证?
 - | └ ☒ 响应式表单
- |
- └ 需要实时数据同步?
 - | └ ☒ 响应式表单



模板驱动表单示例

```

<!-- 简单的登录表单 -->
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm.value)">
  <!-- 文本输入 -->
  <input
    type="email"
    name="email"
    placeholder="邮箱"

```

```

    [(ngModel)]="model.email"
    required
    email
    #emailField="ngModel"
  />
  @if (emailField.invalid && emailField.touched) {
    <div class="error">{{ getEmailError(emailField) }}</div>
  }

  <!-- 密码输入 -->
  <input
    type="password"
    name="password"
    placeholder="密码"
    [(ngModel)]="model.password"
    required
    minlength="8"
    #passwordField="ngModel"
  />

  <!-- 提交按钮 -->
  <button [disabled]="!loginForm.valid">登录</button>
</form>

```

⚙️ 响应式表单深度示例

```

import { FormBuilder, FormGroup, Validators, AbstractControl } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  template: `
    <form [formGroup]="userForm" (ngSubmit)="onSubmit()">
      <!-- 基本字段 -->
      <input
        formControlName="name"
        placeholder="姓名"
      />
      @if (userForm.get('name')?.errors?.['required']) {
        <span class="error">姓名必填</span>
      }

      <!-- 嵌套 FormGroup -->
      <fieldset [formGroup]="userForm.get('address')">
        <input
          formControlName="city"
          placeholder="城市"
        />
      </fieldset>

      <!-- 动态 FormArray -->
      <div formArrayName="hobbies">
        @for (hobby of hobbies().controls; let i = $index) {

```

```

        <div [formGroupName]="i">
            <input formControlName="name" placeholder="爱好名称" />
            <button type="button" (click)="removeHobby(i)">删除</button>
        </div>
    }
</div>
<button type="button" (click)="addHobby()">添加爱好</button>

    <button type="submit" [disabled]="!userForm.valid">保存</button>
</form>
`,
standalone: true,
imports: [ReactiveFormsModule, CommonModule]
})
export class UserFormComponent {
    userForm: FormGroup;

    constructor(private fb: FormBuilder) {
        this.userForm = this.fb.group({
            name: ['', [Validators.required, Validators.minLength(2)]],
            email: ['', [Validators.required, Validators.email]],
            // 嵌套 FormGroup
            address: this.fb.group({
                city: ['',],
                street: ['',],
                zipCode: ['',]
            }),
            // 动态 FormArray
            hobbies: this.fb.array([])
        });

        // 获取 FormArray
        hobbies() {
            return this.userForm.get('hobbies') as FormArray;
        }

        // 添加爱好
        addHobby() {
            const hobbyForm = this.fb.group({
                name: ['', Validators.required]
            });
            this.hobbies().push(hobbyForm);
        }

        // 删除爱好
        removeHobby(index: number) {
            this.hobbies().removeAt(index);
        }

        // 自定义验证器
        passwordMatchValidator(control: AbstractControl): ValidationErrors | null {
            const password = control.get('password')?.value;

```

```
const confirmPassword = control.get('confirmPassword')?.value;

if (password !== confirmPassword) {
  return { passwordMismatch: true };
}
return null;
}

onSubmit() {
  if (this.userForm.valid) {
    console.log(this.userForm.value);
  }
}
}
```

1 1 生命周期钩子完全指南

🔄 生命周期执行顺序图



📊 生命周期钩子详解表

钩子	调用时机	执行次数	用途	优先度
ngOnInit	初始化后	1次	初始化数据、订阅	★★★★★
ngOnDestroy	销毁前	1次	清理资源、取消订阅	★★★★★

钩子	调用时机	执行次数	用途	优先级
<code>ngOnChanges</code>	@Input变化	多次	响应Input变化	★★★★
<code>ngAfterViewInit</code>	视图初始化后	1次	操作@ViewChild	★★★
<code>ngAfterContentInit</code>	内容投影后	1次	操作@ContentChild	★★★
<code>ngDoCheck</code>	变更检测时	多次	自定义检测逻辑	★★
<code>ngAfterViewChecked</code>	视图检查后	多次	● 避免使用	★
<code>ngAfterContentChecked</code>	内容检查后	多次	● 避免使用	★



生命周期最佳实践

```
@Component({...})
export class BestPracticeComponent implements OnInit {
  private readonly destroyRef = inject(DestroyRef);

  constructor(private userService: UserService) {
    // ✗ 不要在这里做复杂初始化
    // ✗ 不要访问 @Input/@ViewChild
  }

  ngOnInit() {
    // ✓ 初始化数据
    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(users => console.log(users));

    // ✓ 订阅
    // ✓ 设置定时器
  }

  ngAfterViewInit() {
    // ✓ 访问 @ViewChild 元素
    // ✓ 操作原生 DOM
  }

  // ✗ 无需 ngOnDestroy – takeUntilDestroyed 自动管理取消订阅

  // 清理定时器/事件监听仍可在 ngOnDestroy 中手动处理
}
```

第三部分：工程实践

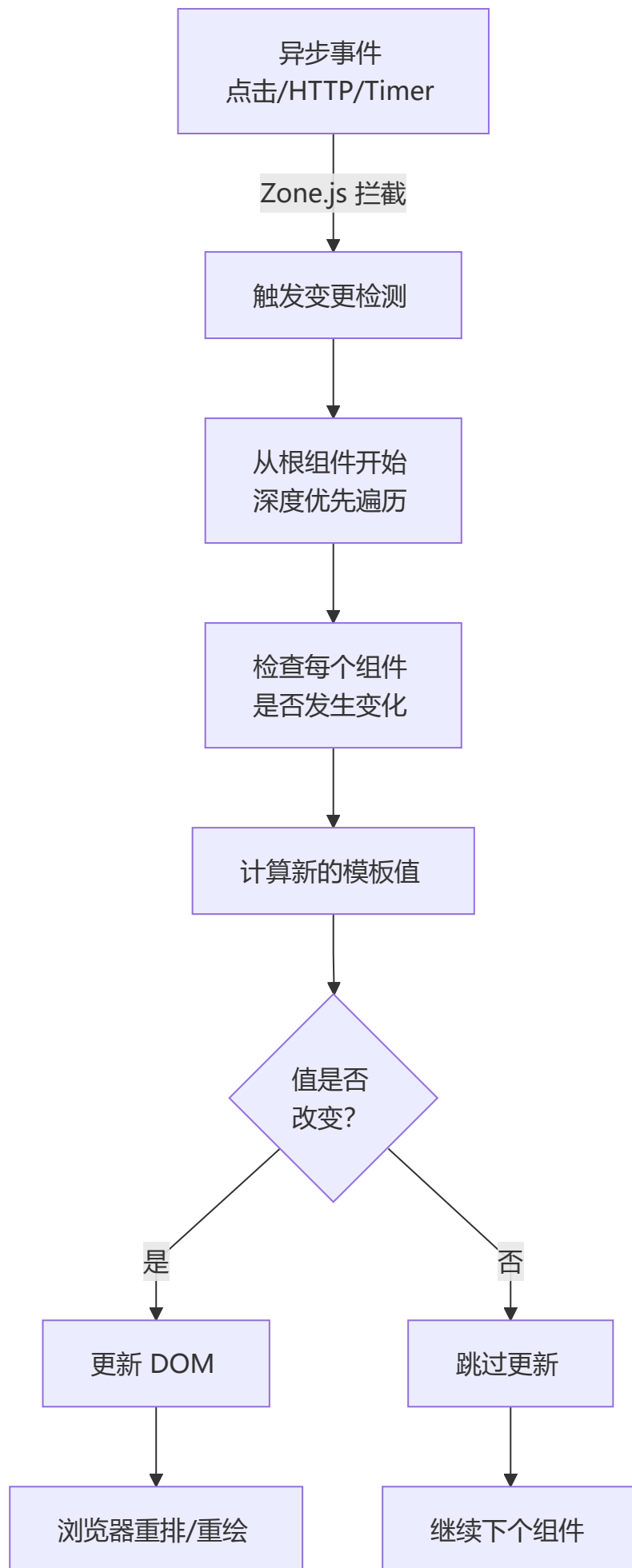
1

2

变更检测机制



变更检测工作原理



📌 ChangeDetectionStrategy

```
// 📌 默认策略：检查整个树
@Component({
  selector: 'app-default',
  template: `<p>{{ data }}</p>`
  // changeDetection: ChangeDetectionStrategy.Default (默认)
})
export class DefaultComponent {
  data = signal('');
}

// 🟢 OnPush 策略：细粒度检测
@Component({
  selector: 'app-onpush',
  template: `<p>{{ user().name }}</p>`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class OnPushComponent {
  user = signal({ name: 'John' });

  // OnPush 何时触发变更检测？
  // 1️⃣ @Input 引用改变
  @Input() set inputData(value: any) {
    // 触发检测
  }

  // 2️⃣ 事件从该组件发出
  onClick() {
    // 点击事件后触发检测
  }

  // 3️⃣ async 管道发出新值
  data$ = this.http.get('/api/data');
  // {{ data$ | async }} 会触发检测

  // 4️⃣ signal 值变化（新特性）
  count = signal(0);
  // {{ count() }} 值变化后触发检测
}
```

🎯 性能优化：OnPush 最佳实践

```
@Component({
  selector: 'app-optimized-list',
  template: `
    @for (item of items; track item.id) {
      <app-list-item
        [item]="item"
        [selected]="item.id === selectedId()"
      >
    `
})
```

```

        (itemClick)="onItemClick($event)"
      />
    }
  },
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class OptimizedListComponent {
  // ✅ 使用 Signal
  items = signal<Item[]>([]);
  selectedId = signal<number | null>(null);

  private cdRef = inject(ChangeDetectorRef);

  // ✅ 不可变更新
  updateItems(newItems: Item[]) {
    this.items.set(newItems); // Signal 自动触发检测
  }

  // ❌ 避免直接修改
  // this.items().push(newItem); ❌ 不会触发检测

  // ✅ 手动触发检测（必要时）
  asyncOperation() {
    this.fetch().subscribe(data => {
      this.items.set(data);
      this.cdRef.markForCheck(); // 标记为脏，下次检测时更新
    });
  }
}

```

1 3 HTTP 和数据获取

HttpClient 完整示例

```

import { HttpClient, HttpResponse, HttpHeaders, HttpParams } from
  '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { retry, catchError, timeout } from 'rxjs/operators';

interface ApiResponse<T> {
  success: boolean;
  data: T;
  message: string;
}

@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = 'https://api.example.com';

  constructor(private http: HttpClient) {}

```

```
// ✅ GET 请求
getUsers(page: number = 1): Observable<User[]> {
  const params = new HttpParams()
    .set('page', page.toString())
    .set('limit', '10');

  return this.http.get<User[]>(`${this.baseUrl}/users`, { params })
    .pipe(
      timeout(5000),           // 5秒超时
      retry(2),               // 失败重试2次
      catchError(this.handleError)
    );
}
```

```
// ✅ POST 请求
createUser(user: Partial<User>): Observable<User> {
  const headers = new HttpHeaders({
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${this.getToken()}`
  });

  return this.http.post<User>(
    `${this.baseUrl}/users`,
    user,
    { headers }
  ).pipe(
    catchError(this.handleError)
  );
}
```

```
// ✅ PUT 请求
updateUser(id: number, updates: Partial<User>): Observable<User> {
  return this.http.put<User>(
    `${this.baseUrl}/users/${id}`,
    updates
  ).pipe(
    catchError(this.handleError)
  );
}
```

```
// ✅ DELETE 请求
deleteUser(id: number): Observable<void> {
  return this.http.delete<void>(
    `${this.baseUrl}/users/${id}`
  ).pipe(
    catchError(this.handleError)
  );
}
```

```
// ✅ 错误处理
private handleError(error: HttpErrorResponse) {
  let errorMessage = '发生了一个错误';
```

```

    if (error.error instanceof ErrorEvent) {
        // 客户端错误
        errorMessage = `错误: ${error.error.message}`;
    } else {
        // 服务器错误
        errorMessage = `错误代码: ${error.status}, 消息: ${error.message}`;
    }

    console.error(errorMessage);
    return throwError(() => new Error(errorMessage));
}
}

```

HTTP 拦截器系统

```

// 📌 认证拦截器
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
    constructor(private authService: AuthService, private router: Router) {}

    intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        // 1 添加 Token
        const token = this.authService.getToken();
        if (token) {
            request = request.clone({
                setHeaders: { Authorization: `Bearer ${token}` }
            });
        }

        // 2 处理响应
        return next.handle(request).pipe(
            catchError(error => {
                if (error.status === 401) {
                    // Token 过期, 重新登录
                    this.authService.logout();
                    this.router.navigate(['/login']);
                }
                return throwError(() => error);
            })
        );
    }
}

// 📌 日志拦截器
@Injectable()
export class LoggingInterceptor implements HttpInterceptor {
    intercept(request: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        const startTime = Date.now();
        console.log(`[${request.method}] ${request.url}`);

        return next.handle(request).pipe(
            tap(event => {

```

```

        if (event instanceof HttpResponse) {
            const duration = Date.now() - startTime;
            console.log(`✅ ${request.method} ${request.url} (${duration}ms)`);
        }
    }
},
catchError(error => {
    const duration = Date.now() - startTime;
    console.error(`❌ ${request.method} ${request.url} (${duration}ms)`);
    return throwError(() => error);
})
);
}
}
}

// 📌 在 main.ts 中注册
bootstrapApplication(AppComponent, {
    providers: [
        provideHttpClient(
            withInterceptors([authInterceptor, loggingInterceptor])
        )
    ]
});

```

🔗 现代方式: `httpResource()`

```

import { resource, httpResource } from '@angular/core/http';

@Component({...})
export class UserListComponent {
    // 📌 使用 httpResource 简化 HTTP 请求
    users = resource({
        request: () => ({ pageSize: 10, page: this.currentPage() }),
        loader: ({ request }) =>
            this.http.get<User[]>('/api/users', {
                params: {
                    pageSize: request.pageSize,
                    page: request.page
                }
            })
    });

    currentPage = signal(1);

    // 自动处理的功能:
    // ✅ 请求状态: users.isLoading
    // ✅ 错误处理: users.error
    // ✅ 数据: users.value
    // ✅ 自动缓存
    // ✅ 自动清理订阅

    onPageChange(page: number) {
        this.currentPage.set(page);
    }
}

```

```
// users 会自动重新加载
}
}
```

第四部分：性能优化

1 4 性能优化全景图

优化策略金字塔



⚡ 包体积优化

```
// 📍 优化前的包体积分析
ng build --stats-json

// 📍 减少依赖
// ❌ 避免导入整个库
import _ from 'lodash'; // 整个库 ~70KB
```

```
// ✅ 只导入需要的部分
import { debounce } from 'lodash-es'; // 只有几KB

// 📍 动态导入（代码分割）
@Component({...})
export class LazyComponent {
  // 使用 import(), 该组件代码不包含在主 bundle 中
}

// 📍 删除未使用的代码
// TreeShaking 要求 package.json 中 sideEffects: false
```

运行时性能优化

```
// ✅ 1. 虚拟滚动（大列表）
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  template: `
    <cdk-virtual-scroll-viewport itemSize="50" class="list">
      @for (item of items; track item.id) {
        <div class="item">{{ item.name }}</div>
      }
    </cdk-virtual-scroll-viewport>
  `
})
export class LargeListComponent {
  items = signal<Item[]>([...Array(10000).keys()].map(i => ({
    id: i,
    name: `Item ${i}`
  })));
}

// ✅ 2. 防抖搜索
@Component({
  template: `
    <input
      #searchInput
      (input)="onSearch(searchInput.value)"
      placeholder="搜索..."
    />
  `
})
export class SearchComponent {
  private searchTerm = signal('');

  onSearch = debounce((term: string) => {
    this.searchTerm.set(term);
    this.performSearch(term);
  }, 300);
}
```



```
// ✅ 3. trackBy 优化列表
@Component({
  template: `
    @for (item of items; track trackById(item)) {
      <app-item [item]="item" />
    }
  `,
})
export class ListComponent {
  items = signal<Item[]>([]);

  trackById(item: Item): number {
    return item.id;
  }
}
```

1 5 测试策略

测试金字塔



单元测试示例

```
import { TestBed } from '@angular/core/testing';
import { signal } from '@angular/core';

describe('用户服务', () => {
  let service: UserService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [UserService],
      imports: [HttpClientTestingModule]
    });

    service = TestBed.inject(UserService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify(); // 确保没有未完成的请求
  });
});
```

```

it('应该获取用户列表', () => {
  const mockUsers: User[] = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' }
  ];

  service.getUsers().subscribe(users => {
    expect(users.length).toBe(2);
    expect(users[0].name).toBe('Alice');
  });

  const req = httpMock.expectOne('/api/users');
  expect(req.request.method).toBe('GET');
  req.flush(mockUsers);
});

it('应该处理错误', () => {
  service.getUsers().subscribe(
    () => fail('应该失败'),
    error => {
      expect(error.status).toBe(404);
    }
  );

  const req = httpMock.expectOne('/api/users');
  req.flush('Not found', { status: 404, statusText: 'Not Found' });
});

// 📁 组件测试
describe('用户列表组件', () => {
  let component: UserListComponent;
  let fixture: ComponentFixture<UserListComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [UserListComponent, HttpClientTestingModule]
    }).compileComponents();

    fixture = TestBed.createComponent(UserListComponent);
    component = fixture.componentInstance;
  });

  it('应该显示用户列表', () => {
    component.users.set([
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' }
    ]);

    fixture.detectChanges();

    const items = fixture.nativeElement.querySelectorAll('.user-item');
  });

```

```
expect(items.length).toBe(2);
expect(items[0].textContent).toContain('Alice');
});
});
```

第五部分：面试题汇总

核心概念面试题

Q1: Angular 中的依赖注入 (DI) 是什么？工作原理如何？

标准答案：

依赖注入是 Angular 的核心特性，它允许组件或服务声明它们需要的依赖项，而不是自己创建它们。

工作流程：

1. 组件/服务声明依赖 → 2. Injector 查询 Provider → 3. 创建实例 → 4. 注入使用

代码示例：

```
// 1 定义服务
@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers() { /* ... */ }
}

// 2 注入到组件
@Component({...})
export class UserListComponent {
  // 方式 A: 构造函数注入（传统）
  constructor(private userService: UserService) {}

  // 方式 B: inject() 函数（现代）
  private userService = inject(UserService);
}

// 3 Angular 自动处理整个过程
```

为什么重要：

- ✅ 解耦：组件不依赖于具体实现
 - ✅ 易测试：可以注入 mock 对象
 - ✅ 单一职责：每个类专注于自己的功能
-

Q2: Signals 和 Observables 的区别？何时使用哪个？

对比表：

方面	Signals	Observables
同步性	✔ 同步	✗ 异步
当前值	✔ 总有值	⚠ 需要订阅
内存开销	低	较高
学习曲线	低	陡峭
库依赖	✗ 无	✔ RxJS

使用指南：

```
// ✔ Signal: 本地组件状态
count = signal(0);
user = signal<User | null>(null);

// ✔ Observable: 异步操作和数据流
users$ = this.http.get('/users');
clicks$ = fromEvent(element, 'click');

// ⚠ 两者结合
userResource = resource({
  request: () => this.filterId(),
  loader: ({ request }) => this.http.get(`/users/${request}`)
});
```

Q3: 如何处理内存泄漏？

常见原因和解决方案：

```
// ✗ 问题 1: 订阅未取消
export class BadComponent {
  constructor(private userService: UserService) {}

  ngOnInit() {
    // ✗ 如果不取消订阅，组件销毁时内存泄漏
    this.userService.users$.subscribe(
      users => console.log(users)
    );
  }
}

// ✔ 解决方案 1: takeUntilDestroyed (Angular 20+ 推荐)
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
```

```

export class GoodComponent {
  private readonly destroyRef = inject(DestroyRef);

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.userService.users$
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(users => console.log(users));
  }
  // 无需 ngOnDestroy
}

// ✅ 解决方案 2: async 管道（自动取消订阅）
@Component({
  template: `<div>{{ users$ | async }}</div>`
})
export class BestComponent {
  users$ = this.userService.users$;
  constructor(private userService: UserService) {}
}

// ✅ 解决方案 3: Signals（不需要取消订阅）
export class ModernComponent {
  users = resource({
    loader: () => this.userService.getUsers()
  });
}

```

Q4: 什么是 OnPush 变更检测策略? 何时应该使用?

工作原理:

Default 策略: 任何变化 → 检查整个树 ❌ 性能差

OnPush 策略: 只在以下情况检查该组件:

- 1 @Input 引用改变
- 2 事件在组件内触发
- 3 signal 值变化
- 4 async 管道发出新值

实战示例:

```

@Component({
  selector: 'app-optimized',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <p>{{ user().name }}</p>
    <button (click)="update()">更新</button>
  `
})

```

```
export class OptimizedComponent {  
  // ✅ 使用 Signal 实现细粒度响应  
  user = signal({ name: 'John' });  
  
  update() {  
    // ✅ Signal 变化会自动触发检测  
    this.user.set({ name: 'Jane' });  
  }  
}
```

性能对比:

Default 策略:

1000 个组件 × 100 次检测 = 100,000 次检查 🐢

OnPush 策略:

实际影响的组件 × 100 次检测 = 1,000 次检查 ⚡

Q5: 如何优化大型 Angular 应用?

优化清单:



包体积优化

- ✅ 延迟加载模块
- ✅ Tree Shaking
- ✅ AOT 编译
- ✅ 代码分割



运行时优化

- ✅ OnPush 策略
- ✅ trackBy
- ✅ 虚拟滚动
- ✅ 防抖/节流



变更检测优化

- ✅ signals 替代 observable
- ✅ 细粒度检测
- ✅ 避免模板中的函数调用



网络优化

- ✅ HTTP 缓存
- ✅ 请求合并
- ✅ CDN 部署

实战场景题

场景1：实现一个具有搜索、排序、分页的数据表格

```
@Component({
  selector: 'app-data-table',
  template: `
    <!-- 搜索框 -->
    <input
      [formControl]="searchControl"
      placeholder="搜索..."
    />

    <!-- 排序选择 -->
    <select (change)="onSortChange($event)">
      <option value="name">按名称排序</option>
      <option value="date">按日期排序</option>
    </select>

    <!-- 数据表格 -->
    <table>
      <thead>
        <tr>
          <th>名称</th>
          <th>日期</th>
          <th>操作</th>
        </tr>
      </thead>
      <tbody>
        @for (item of filteredData(); track item.id) {
          <tr>
            <td>{{ item.name }}</td>
            <td>{{ item.date | date }}</td>
            <td>
              <button (click)="edit(item)">编辑</button>
              <button (click)="delete(item.id)">删除</button>
            </td>
          </tr>
        }
      </tbody>
    </table>

    <!-- 分页 -->
    <div class="pagination">
      <button (click)="previousPage()" [disabled]="currentPage() === 1">
        上一页
      </button>
      <span>第 {{ currentPage() }} 页</span>
      <button (click)="nextPage()">下一页</button>
    </div>
  `,
  standalone: true,
```

```

imports: [CommonModule, ReactiveFormsModule]
})
export class DataTableComponent {
  private dataService = inject(DataService);

  // 响应式状态
  searchControl = new FormControl('');
  sortBy = signal<'name' | 'date'>('name');
  currentPage = signal(1);
  pageSize = 10;

  // 原始数据
  allData = resource({
    loader: () => this.dataService.getData()
  });

  // 搜索过滤
  filteredBySearch = computed(() => {
    const term = this.searchControl.value?.toLowerCase() || '';
    return (this.allData.value() || []).filter(item =>
      item.name.toLowerCase().includes(term)
    );
  });

  // 排序
  sortedData = computed(() => {
    const data = [...this.filteredBySearch()];
    const sortKey = this.sortBy();
    return data.sort((a, b) => {
      if (sortKey === 'name') {
        return a.name.localeCompare(b.name);
      } else {
        return new Date(a.date).getTime() - new Date(b.date).getTime();
      }
    });
  });

  // 分页
  filteredData = computed(() => {
    const start = (this.currentPage() - 1) * this.pageSize;
    const end = start + this.pageSize;
    return this.sortedData().slice(start, end);
  });

  onSortChange(event: Event) {
    const select = event.target as HTMLSelectElement;
    this.sortBy.set(select.value as 'name' | 'date');
    this.currentPage.set(1);
  }

  previousPage() {
    if (this.currentPage() > 1) {
      this.currentPage.update(p => p - 1);
    }
  }
}

```



```

    }
  }

  nextPage() {
    this.currentPage.update(p => p + 1);
  }

  edit(item: DataItem) {
    // 编辑逻辑
  }

  delete(id: number) {
    this.dataService.deleteItem(id).subscribe(() => {
      // 刷新数据
    });
  }
}

```

代码质量

如何组织 Angular 项目结构？

推荐的项目结构：

```

src/
├── app/
│   ├── core/                                # 核心模块（单例服务）
│   │   ├── services/
│   │   │   ├── auth.service.ts
│   │   │   └── api.service.ts
│   │   ├── interceptors/
│   │   ├── guards/
│   │   └── core.module.ts
│   │
│   ├── shared/                              # 共享模块（可复用组件）
│   │   ├── components/
│   │   │   ├── header/
│   │   │   ├── footer/
│   │   │   └── loading/
│   │   ├── pipes/
│   │   ├── directives/
│   │   └── shared.module.ts
│   │
│   ├── features/                            # 功能模块
│   │   ├── dashboard/
│   │   │   ├── dashboard.component.ts
│   │   │   ├── dashboard.routes.ts
│   │   │   └── services/
│   │   ├── products/
│   │   │   ├── product-list/
│   │   │   └── product-detail/

```

```
| | | └─ services/
| | └─ admin/
| |
| └─ app.routes.ts          # 路由配置
| └─ app.component.ts      # 根组件
| └─ app.config.ts         # 应用配置
|
└─ assets/                 # 静态资源
    └─ images/
    └─ styles/
    └─ data/
```

核心原则：

- ✅ 单一职责：每个文件一个功能
- ✅ 可扩展性：易于添加新功能
- ✅ 可维护性：代码结构清晰
- ✅ 可测试性：便于单元测试
- ✅ 可复用性：共享组件集中管理

性能指标

如何衡量 Angular 应用的性能？

```
// 🚦 关键性能指标 (Core web vitals)

// 1 LCP (Largest Contentful Paint) - 最大内容绘制
// ✅ 目标: < 2.5 秒
// 优化: 预加载资源、Code Splitting

// 2 FID (First Input Delay) - 首次输入延迟
// ✅ 目标: < 100 毫秒
// 优化: 减少主线程工作、使用 web workers

// 3 CLS (Cumulative Layout Shift) - 累积布局偏移
// ✅ 目标: < 0.1
// 优化: 预留尺寸空间、避免突然 DOM 插入

// 📍 性能监控代码
export class PerformanceService {
  logNavigationTiming() {
    window.addEventListener('load', () => {
      const perfData = window.performance.timing;
      const pageLoadTime = perfData.loadEventEnd - perfData.navigationStart;
      console.log(`页面加载时间: ${pageLoadTime}ms`);
    });
  }
}

logCorewebvitals() {
  // 使用 web-vitals 库
```

```
import('web-vitals').then(({ getLCP, getFID, getCLS }) => {  
  getLCP(console.log);  
  getFID(console.log);  
  getCLS(console.log);  
});  
}  
}
```

总结与最佳实践

Angular 开发黄金法则

- 1 优先使用 **signals** 进行状态管理
→ 更简洁、更高效、更易理解
- 2 默认采用 **OnPush** 变更检测
→ 性能提升 20-30%
- 3 响应式表单优于模板驱动表单
→ 复杂表单首选
- 4 始终在 **ngOnDestroy** 中清理资源
→ 防止内存泄漏
- 5 优先 **async** 管道处理 **observables**
→ 自动管理订阅
- 6 使用 **trackBy** 优化列表性能
→ 避免不必要的 **DOM** 操作
- 7 类型安全始终第一
→ 充分利用 **TypeScript**
- 8 分离关注点
→ 每个组件/服务单一职责
- 9 编写可测试的代码
→ 提高代码质量和维护性
- 10 遵循 **Angular** 风格指南
→ 保持代码一致性



推荐学习资源

-  [官方文档](#)
-  [Angular 风格指南](#)
-  [Angular University](#)

-  [StackBlitz 在线编辑器](#)

如有问题或建议，欢迎提出! 🚀